Vigil MDR Implementation Guide

How the Vigil MDR lab project is built, how it runs, and what remains. Version of 31 October 2026.

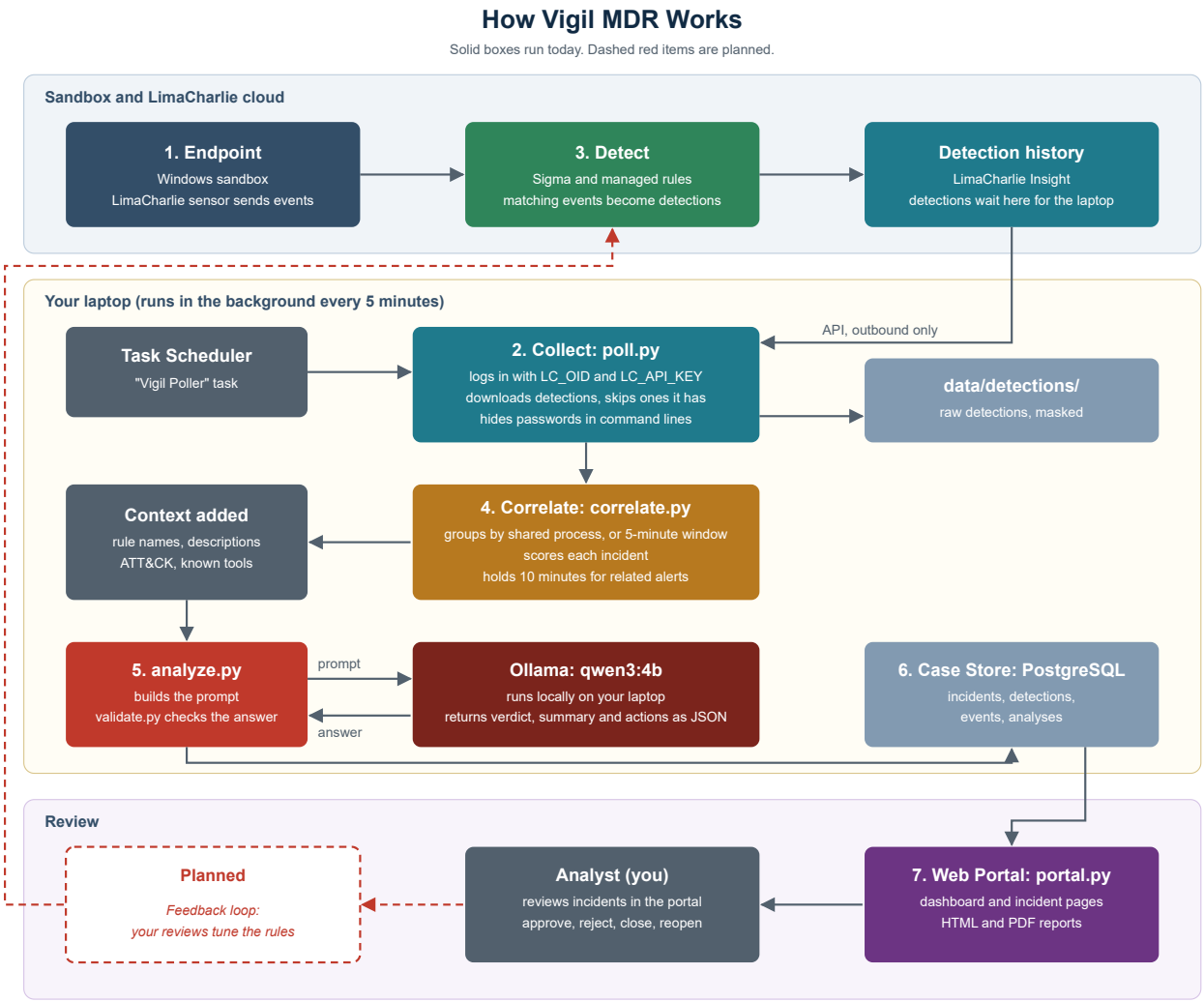
1. Overview

Vigil is a personal Managed Detection and Response lab. It collects detections from LimaCharlie, groups related alerts into scored incidents, asks a local AI model for a triage opinion, stores the results in PostgreSQL, and gives an analyst a web portal to review them.

It's a learning project, not a production security service. It doesn't block attacks, and a person makes the final decision on every incident.

> **Current limitation:** AI verdicts aren't reliable yet and accuracy hasn't been measured. In testing, two nearly identical incidents received opposite verdicts. Section 8 has the details.

2. Architecture



Stage	Component	Status
1. Endpoints	Windows sandbox with the LimaCharlie sensor	Working
2. Collect	`poll.py`, run every 5 minutes	Working
3. Detect	LimaCharlie Sigma and managed rules	Working, needs tuning
4. Correlate	`correlate.py`	Working, needs tuning
5. AI Analyze	`analyze.py`, `build_prompt.py`, Ollama	Running, accuracy not measured
6. Case Store	PostgreSQL 17 through `db.py`	Working
7. Web Portal	`portal.py`, FastAPI, Jinja, HTMX	Working
Analyst review	Approve, reject, close and reopen	Working
Feedback loop	Reviews tune detection rules	Planned

Key design decisions

- **Polling replaced the webhook.** The laptop makes outbound requests only, and detections missed while it's off are collected from LimaCharlie's history on the next run.
- **The AI runs locally.** Detection data is sent to Ollama on the laptop, not to an external AI service.
- **Code owns the facts.** Grouping, scoring, ATT&CK techniques and timelines come from code. The model only provides an opinion.
- **Evidence is untrusted.** Command lines can be written by an attacker, so they stay separate from trusted context and are escaped on every page.

3. Project layout

Path	Purpose
`poll.py`	Main pipeline: fetch, mask, save, correlate, analyze, store
`adapters/limacharlie.py`	Converts a detection into a simple incident shape and masks secrets
`correlate.py`	Groups detections and calculates scores
`analyze.py`	Calls Ollama, retries once, and defangs the answer
`build_prompt.py`	Builds the prompt from core rules and matching playbooks
`validate.py`	Checks the schema and rejects invented event IDs
`defang.py`	Makes IPs, domains, URLs and emails unclickable
`db.py`, `schema.sql`	PostgreSQL storage and table definitions
`portal.py`	Web portal, login and security controls
`process_tree.py`	Builds process trees from LimaCharlie process links
`report.py`	Builds the HTML and PDF reports

`prompts/`	`core.md` and 11 playbooks
`schema/verdict.schema.json`	Required format for AI answers
`templates/`, `static/`	Portal pages, styling, logo and HTMX
`tools/`	Daily status and portal password scripts
`data/`	Local detections, logs and reports, excluded from Git

4. Data flow

1. The Windows task "Vigil Poller" runs `poll.py` every 5 minutes. A new run doesn't start while one is active.
2. `poll.py` exchanges `LC_OID` and `LC_API_KEY` for a short-lived token, then requests detections since the last check, plus 10 extra minutes. It follows `next_cursor` through all pages.
3. Passwords and tokens in argument values such as `--auth=` are replaced with `[REDACTED]`, then new detections are saved to `data/detections/`.
4. All saved detections are regrouped into incidents and synced to PostgreSQL.
5. An incident becomes ready 10 minutes after its last detection, so related alerts can join it.
6. Up to 10 incidents per run go to Ollama, highest score first. An incident is analyzed again if its detections change.
7. The verdict is validated, defanged and stored. If Ollama is unreachable, the run stops and a later run retries.

5. Correlation and scoring

Grouping rules

Rule	Value
Scope	Same host
Shared process link	Up to 30 minutes apart
Timing only	Up to 5 minutes apart, not for known tools
Maximum incident span	1 hour
Prompt limits	12 unique events, command lines cut at 400 characters, 6 rules

Process links use LimaCharlie atoms, which are unique IDs. They're more reliable than process IDs, which Windows reuses.

Score

Factor	Effect
Base	Highest rule severity in the incident
Several ATT&CK techniques	+5 per extra technique, up to +15

Several rules	+5 per extra rule, up to +10
LSASS targeted	+20
Known tool involved	-30

Scores stay between 0 and 100. A score measures alert severity, not the likelihood of an attack.

6. AI analysis

The model is `qwen3:4b`, called through Ollama's local API with temperature 0, an 8192-token context and JSON output forced to match the schema.

1. `build_prompt.py` maps the incident's ATT&CK techniques to playbooks and keeps at most 4 for a normal analysis.
2. Trusted data goes in a `<context>` block, and events go in `<evidence>`.
3. `validate.py` checks the schema and every cited event ID in claims and the attack chain.
4. On failure, the errors are sent back for one retry. If it still fails, the incident is marked `failed`.
5. `defang.py` masks indicators in the answer, even if the model missed them.

The answer contains a verdict, confidence, severity, summary, attack chain, claims, indicators, evidence requests and recommended actions.

7. Database

Table	Holds
`incidents`	Host, score, time range, grouping method and context
`detections`	Masked detections, rule, time, ATT&CK techniques and incident link
`events`	Process events used as evidence
`analyses`	Every AI verdict, including older versions
`reviews`	Analyst decisions. These have no foreign key, so they survive regrouping

PostgreSQL 17 is installed from conda-forge in `F:\PostgreSQL`, because the official download server refused the connection and drive C: was low on space. It accepts local connections only, and the application uses a dedicated `vigil` login.

8. Testing and known limits

Verified

- The saved detection count matched LimaCharlie's 7-day history, with none missing.
- Signed-out visitors can't open portal pages, and forged cookies, invalid hosts and cross-site form posts are refused.
- Injected script text displays harmlessly in the portal and process tree.

- Unmasked `--auth=` values weren't found in saved detections or reports.

AI test results

Test	Result
First Orborus and LSASS incident	`benign`, high confidence, with an unsupported claim that LSASS wasn't accessed
Second Orborus and LSASS incident	`true_positive`, critical, with an unsupported claim of credential theft. Rejected for citing an invented event ID
Speed	About 3 to 7 minutes per incident with the full prompts

Known limits

- The model may be too small to apply the full guidance reliably.
- The laptop is the server, so the site and analysis stop when it's off or asleep.
- The portal and tunnel don't start by themselves after a restart.
- The quick tunnel's public address changes on restart and is meant for testing.
- The portal has a single password and no two-step sign-in.
- The database has no backups yet.
- Automatic masking only covers common argument names.
- Process trees only include processes that appear in detections.

9. Portal security

Control	Implementation
Login	Every page except login and static files needs a password
Password storage	Salted scrypt hash in a Windows user variable
Sessions	Signed cookie, HttpOnly, SameSite Strict, Secure in public mode, 8-hour limit
Guessing	5 failed attempts blocks an address for 15 minutes
Forged requests	Forms must come from the portal's own origin
Hosts	Only local names and `*.trycloudflare.com` are accepted
Page safety	Automatic escaping and a Content Security Policy
Network	Portal listens on 127.0.0.1, with public HTTPS through the tunnel

10. Operations

Item	Value
Scheduled tasks	"Vigil Poller", "Vigil PostgreSQL", "Vigil Daily Status"

User variables	`LC_OID`, `LC_API_KEY`, `VIGIL_DB_PASSWORD`, `VIGIL_PORTAL_HASH`, `VIGIL_SESSION_SECRET`
Logs	`data\poll.log`, `data\portal.log`, `F:\PostgreSQL\postgres.log`
Reports	`data\reports\`
Setup and daily use	`instructions.md`

> Never commit `data/`, `.venv/`, API keys or passwords. Reports and detections contain hostnames, usernames, IP addresses and command lines.

11. Remaining work

1. Add code rules for known tools touching LSASS, and block contradictory verdicts.
2. Test a larger Ollama model if the laptop can run it.
3. Find out whether Orborus access to LSASS is normal.
4. Build an eval set and measure verdict accuracy.
5. Tune grouping, scoring and noisy detection rules.
6. Set up database backups, and confirm services recover after a restart.
7. Decide how the portal and tunnel should start by themselves.
8. Build the feedback loop from reviews into rule tuning.

The live checklist is in `SPRINT_STATUS.md`.